

Compte rendu : Projet RAG — Recommandation de Films

L'objectif est de construire un assistant capable de recommander des films à partir du dataset TMDb 5000 Movie Dataset, en utilisant un pipeline RAG avec Python, FAISS, sentence-transformers et Groq.

Questions de réflexion préliminaires :

Q1. Comment convertir les données tabulaires en texte cohérent à embedder ?

Les données sont sous forme de CSV, donc chaque ligne représente un film. Pour permettre au modèle d'embedding de comprendre correctement chaque film, je vais transformer chaque ligne en un texte structuré.

Q2. Comment extraire proprement la colonne genres ?

La colonne `genres` est au format JSON imbriqué.

Je vais utiliser le module Python json avec json.loads() pour convertir cette chaîne de caractères en liste Python, puis récupérer uniquement la valeur du champ name.

Q3. Quelle stratégie pour éviter de relancer l'indexation à chaque test ?

L'indexation peut prendre du temps car il faut transformer plusieurs milliers de films en embeddings.

Pour éviter de refaire cette étape à chaque lancement, je vais sauvegarder :

- l'index FAISS dans un fichier. index
- les textes et métadonnées dans un fichier. json

Ainsi, indexation.py sera exécuté seulement une fois pour créer la base vectorielle. Ensuite, rag.py chargera directement l'index déjà sauvegardé.

Q4. Comment guider le LLM pour produire de bonnes recommandations ?

Le prompt système devra préciser que le LLM est un assistant de recommandation de films.

Il devra :

- recommander uniquement des films présents dans le contexte fourni
- expliquer pourquoi chaque film correspond à la demande
- citer le titre du film
- citer la note du film
- ne pas inventer de film absent de la base
- proposer plusieurs recommandations quand c'est pertinent

Comme la recommandation n'a pas une seule bonne réponse, le LLM devra justifier ses choix avec les informations disponibles : genre, description, année, note et langue.

Q5. Que faire si l'utilisateur demande un film très récent absent de la base ?

Si l'utilisateur demande un film de 2024 ou un film absent de la base, le système ne doit pas inventer.

Il doit répondre clairement :

"Je ne trouve pas cette information dans ma base de connaissance."

Il peut ensuite proposer des films similaires présents dans la base, si le contexte le permet.

Etape 1 : Préparation des données

Les champs les plus importants à inclure dans le texte à embedder sont :

"Title, overview, genres, release_date, vote_average, original_language. "

Le champ `overview` est essentiel car il contient le synopsis du film et permet au modèle d'embedding de comprendre le thème, l'histoire et l'univers du film.

Les genres permettent d'améliorer la recherche par type de film (science-fiction, romance, action, etc.).

Le titre aide le système à reconnaître directement certains films connus.

La date de sortie, la note et la langue originale ajoutent du contexte supplémentaire utile pour les recommandations.

Certaines informations sont surtout utiles pour l'affichage final ou le filtrage, mais ne sont pas forcément nécessaires dans les embeddings.

Les champs gardés en métadonnées sont :

title, release_date , vote_average, original_language, genres.

Ces métadonnées permettent :

- d'afficher les informations des films dans les résultats
- de montrer les sources utilisées
- de comparer deux films
- d'afficher les scores et détails des recommandations

Certaines données nécessitent un nettoyage avant la vectorisation.

Étape 2 - Chunking

Notre contenu est composé de synopsis de films accompagnés de quelques informations courtes comme le titre, la date de sortie, la note, la langue originale et les genres. Ces textes sont relativement courts. Il n'est donc pas nécessaire de les découper en plusieurs morceaux. Nous avons choisi de garder un film complet dans un seul chunk afin de conserver tout le contexte du film.

Le contenu est structuré ligne par ligne avec des retours à la ligne : Title, Release date, Overview, Rating, Original language et Genres. Le séparateur naturel est donc le retour à la ligne \n, car il permet de garder une structure claire et lisible.

Aucun overlap n'est nécessaire dans notre cas. Comme chaque film est court et traité comme un seul chunk, il n'y a pas de perte d'information entre deux morceaux. Ajouter un overlap créerait seulement de la répétition inutile.

Nous avons testé le chunker sur un document. Le résultat montre un seul chunk contenant toutes les informations du film, ce qui confirme que la stratégie choisie est adaptée à notre corpus.

```
Test du chunking sur un document :

--- Chunk 400 ---
Title: Divergent
Release date: 2014-03-14
Overview: In a world divided into factions based on personality types, Tris learns that she's been classified as Divergent and won't fit in. When she discovers a plot to destroy Divergents, Tris and the mysterious Four must find out what makes Divergents dangerous before it's too late.
Rating: 6.9/10
Original language: en
Genres: Adventure, Action, Science Fiction
```

Étape 3 - Création des embeddings

Le contenu est principalement en anglais, car les titres, synopsis et métadonnées du dataset TMDb sont en anglais. J'ai donc choisi le modèle all-MiniLM-L6-v2, adapté à l'encodage de textes courts en anglais.

Pour afficher une barre de progression pendant l'encodage, j'utilise le paramètre `show_progress_bar=True` dans la méthode `model.encode()`.

Pour vérifier que les vecteurs ont la bonne dimension, j'affiche `embeddings.shape`. Le résultat obtenu est `(4803, 384)`, ce qui signifie que 4803 documents ont été encodés, chacun avec un vecteur de dimension 384.

```
Nombre de vecteurs : 4803
Dimension des vecteurs : 384
Shape embeddings : (4803, 384)
```

Étape 4 - Création et persistance de la base FAISS

Il est important de sauvegarder les chunks dans le même ordre que les vecteurs FAISS, car FAISS retourne uniquement les indices des vecteurs les plus proches. Par exemple, si FAISS retourne l'indice 10, on doit pouvoir retrouver le chunk correspondant avec `documents[10]`. Si l'ordre change, le résultat de recherche ne correspondra plus au bon film.

`IndexFlatL2` utilise la distance euclidienne pour comparer les vecteurs. `IndexFlatIP` utilise le produit scalaire entre les vecteurs. Pour utiliser une similarité cosinus, il faut normaliser les vecteurs, puis utiliser `IndexFlatIP`. Sans normalisation, `IndexFlatIP` ne correspond pas exactement à la similarité cosinus.

Pour vérifier que l'index a bien été sauvegardé et rechargé, on sauvegarde l'index avec `faiss.write_index(index, "index.faiss")`, puis on le recharge avec `faiss.read_index("index.faiss")`. Ensuite, on affiche le nombre de vecteurs contenus dans l'index rechargé avec `index_recharge.ntotal`. Dans notre cas, le résultat est 4803, ce qui confirme que l'index a bien été sauvegardé et rechargé correctement.

Étape 5 - Fonction de recherche

Le score retourné par FAISS correspond à une distance entre le vecteur de la requête et les vecteurs des documents.

Dans le cas de l'index utilisé (`IndexFlatL2`), il s'agit d'une distance euclidienne. Plus cette distance est faible, plus le document est proche de la requête et donc pertinent. À l'inverse, un score élevé signifie que le document est moins pertinent.

La normalisation des vecteurs n'est pas obligatoire avec `IndexFlatL2`, car cet index utilise une distance euclidienne. Cependant, elle devient nécessaire si l'on souhaite utiliser la similarité cosinus. En normalisant les vecteurs (norme = 1), la similarité cosinus peut être utilisée via un produit scalaire, ce qui permet de mieux comparer le sens des textes.

Pour tester la pertinence de la fonction de recherche, il suffit de poser plusieurs questions et d'analyser les résultats retournés. Il faut vérifier que les films ou documents trouvés sont cohérents avec le sens de la question.

Par exemple, une requête sur une histoire d'amour compliquée doit retourner des films de romance ou de drame.

Si les résultats sont cohérents, cela signifie que la recherche fonctionne correctement.

Exemple de test : question : sad drama movie

Les résultats obtenus sont globalement cohérents avec les thématiques demandées, ce qui valide la qualité du système de recherche.

Étape 6 - Génération de la réponse avec Groq

Il faut présenter les chunks de manière structurée et lisible. Il est préférable de les numéroté afin que le LLM puisse distinguer chaque document. Il est aussi utile d'inclure les métadonnées importantes comme le titre, les genres, la note et la date de sortie. Cela permet au LLM de générer une réponse plus claire et plus fiable.

Si on envoie trop de chunks, le prompt devient trop long et peut dépasser la taille maximale du contexte du modèle. Le LLM peut alors ignorer une partie des informations, ralentir, coûter plus cher ou retourner une réponse moins précise. Il faut donc limiter le nombre de chunks envoyés, par exemple avec les 3 à 5 résultats les plus pertinents.

On force Llm avec un prompt système clair. Par exemple, on lui indique qu'il doit répondre uniquement avec les films présents dans le contexte, ne pas inventer de films, et dire qu'il n'a pas assez d'informations si le contexte ne permet pas de répondre. Cela réduit les hallucinations.

Dans le TP, les modèles mentionnés sont llama3-8b-8192 et llama3-70b-8192. Le modèle 8B est plus rapide et suffisant pour un prototype ou des réponses simples. Le modèle 70B est généralement meilleur pour des réponses plus détaillées, mais il est plus lent et plus coûteux. Pour ce projet de recommandation de films, un modèle 8B suffit au départ.

Un test avec une requête hors domaine montre que le système ne retourne pas de faux résultats.